NOTE MÉTHODOLOGIQUE

Projet 7 – Implémentation d'un modèle de scoring crédit

Prêt à dépenser

Champ	Valeur
Auteur	Stéphane BARRÉ
Formation	Data Scientist – OpenClassrooms
Date	Mars 2026
Version	1.0 – Version soutenance
Modèle	LightGBM – AUC 0.787
Déploiement	FastAPI (Render) + Streamlit Cloud

1. Contexte et problématique

La société Prêt à dépenser propose des crédits à la consommation à des personnes disposant de peu ou pas d'historique bancaire. L'entreprise souhaite automatiser l'octroi de crédit en s'appuyant sur un algorithme de scoring, tout en garantissant la transparence des décisions vis-à-vis des clients.

L'objectif de ce projet est de construire un pipeline complet, du prétraitement à la mise en production, incluant :

- Un modèle de classification binaire (défaut / non-défaut)
- Une optimisation du seuil de décision selon une fonction de coût métier
- Une API de prédiction déployée en production (FastAPI sur Render)
- Un dashboard interactif explicable pour les chargés de clientèle (Streamlit Cloud)
- Un système de monitoring de la dérive des données (Evidently AI)
- Un tracking des expérimentations (MLflow) pour la comparaison et la reproductibilité des modèles

2. Description des données

Le dataset utilisé est le Home Credit Default Risk (Kaggle). Il comprend 307 511 clients décrits dans 7 tables relationnelles.

Table	Description	Lignes	Colonnes
application_train	Table principale – demandes de crédit	307 511	122
bureau	Historique des crédits auprès d'autres banques	1 716 428	17
bureau_balance	Soldes mensuels des crédits bureau	27 299 925	3
previous_application	Demandes précédentes chez Prêt à dépenser	1 670 214	37
POS_CASH_balance	Soldes mensuels POS et cash	10 001 358	8
credit_card_balance	Soldes mensuels cartes de crédit	3 840 312	23
installments_payments	Historique des paiements échelonnés	13 605 401	8

La variable cible TARGET est binaire : 1 = défaut de paiement (8,07 %), 0 = remboursement normal (91,93 %). Ce fort déséquilibre de classes a nécessité une stratégie dédiée.

3. Feature engineering et agrégation

La table principale (application_train) contient 122 colonnes. Pour enrichir la représentation de chaque client, nous avons agrégé les 6 tables secondaires à l'aide de statistiques descriptives (mean, sum, max, min, std, count) appliquées aux variables numériques les plus pertinentes.

3.1 Stratégie d'agrégation

- Agrégation de bureau et bureau_balance : historique crédit externe
- Agrégation de previous_application : comportement passé chez Prêt à dépenser
- Agrégation de POS_CASH_balance, credit_card_balance, installments_payments : comportement de remboursement

Chaque table secondaire est agrégée par SK_ID_CURR puis jointurée à la table principale. Les colonnes résultantes sont préfixées par le nom de la table source (ex. BUREAU_AMT_CREDIT_SUM_mean).

3.2 Résultat

Étape	Nombre de features
Table principale uniquement	122
Après agrégation des 6 tables secondaires	804
Gain net	+682 features engineered

Ce feature engineering intensif est l'un des leviers principaux du score AUC obtenu (0.787). Il permet au modèle de capturer la complexité du comportement de remboursement sur plusieurs horizons temporels.

4. Prétraitement et pipeline

4.1 Nettoyage

- Suppression des colonnes avec plus de 60 % de valeurs manquantes
- Remplacement des valeurs aberrantes identifiées (DAYS_EMPLOYED = 365243)
- Encodage des variables catégorielles binaires (CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY)

4.2 Pipeline scikit-learn

Un pipeline scikit-learn a été construit pour garantir la reproductibilité et éviter tout data leakage :

- Imputation par la médiane pour les variables numériques
- Imputation par la valeur la plus fréquente pour les variables catégorielles
- OneHotEncoder pour les variables catégorielles multi-classes
- Le pipeline est sauvegardé en preprocesseur.joblib et rechargé à l'identique en production

4.3 Split train/test

Le dataset est découpé en 80 % entraînement / 20 % validation avec stratification sur la variable cible, garantissant la représentation du déséquilibre dans les deux ensembles.

5. Traitement du déséquilibre des classes

Avec seulement 8,07 % de défauts, un modèle naïf atteint 92 % d'accuracy en prédisant toujours 0, ce qui est sans valeur métier. Deux approches ont été comparées :

Méthode	Recall classe 1	AUC-ROC	Temps	Décision
SMOTE (suréchantillonnage)	~0.55	~0.775	~45 min	Rejeté
class_weight='balanced'	~0.58	0.787	~8 min	Retenu □

La méthode class_weight='balanced' pondère automatiquement les erreurs sur la classe minoritaire (facteur ~11x), sans générer de données synthétiques. Elle est plus stable, plus rapide, et produit un meilleur AUC sur ce dataset.

6. Modélisation

6.1 Comparaison des modèles

Modèle	AUC-ROC	Recall cl.1	Temps
Régression Logistique	0.721	0.52	< 1 min
Random Forest	0.762	0.54	~30 min
XGBoost	0.779	0.55	~15 min
LightGBM	0.787	0.58	~8 min

6.2 Justification du choix LightGBM

- Meilleur AUC-ROC (0.787) et meilleur Recall classe 1 (58 %)
- Temps d'entraînement réduit (GBDT avec feuilles plutôt que niveaux)
- Gestion native des valeurs manquantes
- Support natif de class_weight
- Hyperparamètres optimisés par validation croisée 5 folds

6.3 Tracking des expérimentations avec MLflow

Conformément aux exigences de l'énoncé du Projet 7, les expérimentations de modélisation ont été trackées avec MLflow dans le Notebook 03 (Comparaison des Modèles). MLflow

permet de logger automatiquement les paramètres, les métriques et les artefacts de chaque run, facilitant la comparaison et la reproductibilité des expériences.

Configuration MLflow

- Expérience configurée : `P7_Comparaison_Modeles`
- 5 runs comparatifs : DummyClassifier, LogisticRegression, RandomForest, XGBoost, LightGBM
- Lancement de l'interface : `mlflow ui --port 5000` → http://localhost:5000

Ce qui est enregistré par run

Catégorie	Détail
Paramètres	n_estimators, learning_rate, num_leaves, max_depth, class_weight
Métriques	auc_roc, recall_class1, precision_class1, f1_class1, train_time_sec
Artefacts	Modèle sérialisé (mlflow.sklearn / mlflow.lightgbm)
Tags	model_type, dataset, n_features (804), n_clients_train, class_imbalance

7. Fonction de coût métier et optimisation du seuil

7.1 Définition du coût métier

La métrique d'évaluation standard (accuracy) ne reflète pas les enjeux financiers. Un faux négatif (accorder un crédit à un client qui va faire défaut) est bien plus coûteux qu'un faux positif (refuser un bon client).

Nous avons retenu le ratio 10:1 (Baesens et al., 2003), correspondant aux pratiques standard du secteur :

Type d'erreur	Description	Coût
Faux Négatif (FN)	Client à risque accepté → défaut de paiement	10 unités
Faux Positif (FP)	Bon client refusé → perte d'opportunité	1 unité

Formule : Coût total = 10 × FN + 1 × FP (à minimiser)

7.2 Optimisation du seuil

Le seuil de décision par défaut (0.50) n'est pas optimal pour cette fonction de coût asymétrique. Nous avons testé 101 seuils de 0.01 à 0.99 et calculé le coût métier normalisé pour chacun.

Paramètre	Seuil par défaut (0.50)	Seuil optimal (0.370)

Recall classe 1	~42 %	~58 %
Faux positifs	Moins nombreux	Plus nombreux (acceptable)
Coût métier normalisé	Baseline	-2.18 % (amélioration)
Faux négatifs évités	–	~827 de moins

Le seuil optimal de 0.370 permet de détecter davantage de défauts (recall +16 %) en acceptant légèrement plus de refus erronés. C'est le compromis optimal compte tenu du ratio de coût 10:1.

8. Tableau de synthèse des résultats

Métrique	Valeur obtenue	Commentaire
AUC-ROC	0.787	Top performances Kaggle sur ce dataset
Recall classe 1	~58 %	Après optimisation du seuil
Seuil de décision	0.370	Optimisé vs 0.50 par défaut
Amélioration coût métier	+2.18 %	Économie de 1 445 unités de coût sur validation
Nombre de features	804	Après agrégation des 6 tables
Taille du dataset	307 511 clients	Dataset complet Home Credit
Temps d'inférence	< 500 ms	Sur Render (gratuit)

9. Interprétabilité du modèle (SHAP)

9.1 Interprétabilité globale

Les valeurs SHAP (SHapley Additive exPlanations) permettent de mesurer la contribution de chaque feature à la prédiction. Les 5 features les plus importantes du modèle sont :

- EXT_SOURCE_2 et EXT_SOURCE_3 : scores externes de solvabilité (contribution négative = réduction du risque)
- DAYS_BIRTH : âge du client (les clients plus âgés sont plus fiables)
- AMT_CREDIT : montant du crédit demandé
- DAYS_EMPLOYED : ancienneté dans l'emploi (corrélée négativement avec le risque)

9.2 Interprétabilité locale

Pour chaque client, un graphique SHAP waterfall explique les facteurs ayant conduit à la décision :

- Barres rouges : features augmentant la probabilité de défaut
- Barres vertes : features réduisant la probabilité de défaut
- Valeur de base : probabilité moyenne de défaut sur le dataset (~8 %)

Cette approche garantit la conformité avec les exigences de transparence algorithmique (RGPD Art. 22, directive sur le crédit à la consommation).

10. Analyse de la dérive des données (Data Drift)

Le monitoring de la dérive des données est réalisé avec la bibliothèque Evidently AI. Un rapport HTML interactif est généré en comparant la distribution des features entre le jeu d'entraînement et un échantillon de validation.

Indicateur	Résultat
Méthode de détection	Test de Kolmogorov-Smirnov (variables continues)
Seuil de détection	p-value < 0.05
Features analysées	804 features
Features en dérive	< 5 % (dataset stable)
Conclusion	Pas de dérive significative sur le jeu de validation

En production, ce rapport serait généré périodiquement (hebdomadaire) pour détecter toute évolution de la population cliente et déclencher un réentraînement si nécessaire.

11. Limites et améliorations possibles

11.1 Limites actuelles

- Le ratio de coût 10:1 est une hypothèse simplificatrice – en production, il devrait être calibré sur les coûts financiers réels de l'entreprise
- Le dashboard utilise un formulaire simplifié (~50 features) pour les saisies manuelles, les 754 features manquantes étant imputées à zéro
- Le modèle n'intègre pas de features temporelles avancées (tendances, saisonnalité)
- L'infrastructure Render gratuite entraîne une latence au démarrage à froid (~30 s)

11.2 Améliorations envisagées

- Calibration du ratio de coût avec les données financières réelles
- Test A/B du seuil 0.370 vs 0.500 sur 3-6 mois en production
- Intégration de features temporelles (rolling windows, tendances de remboursement)
- Pipeline de réentraînement automatique déclenché par le monitoring de dérive

- Ajout d'un système de feedback pour améliorer le modèle avec les résultats réels

12. Conclusion

Ce projet démontre la mise en œuvre complète d'un système de scoring crédit, de l'exploration des données à la mise en production. Les points différenciants sont :

- 804 features engineered depuis 7 tables relationnelles (vs ~122 en moyenne)
- Optimisation du seuil de décision par fonction de coût métier asymétrique (10:1)
- Interprétabilité locale SHAP intégrée dans l'API et le dashboard
- Déploiement cloud complet avec CI/CD automatisé (GitHub Actions)
- Monitoring de la dérive des données avec rapport HTML automatique
- Tracking MLflow complet (5 runs, métriques, artefacts) assurant la traçabilité et la reproductibilité des expériences

Le système est opérationnel et accessible en production. Il répond aux exigences de transparence algorithmique et constitue une base solide pour un déploiement métier réel.

Références bibliographiques

- Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., & Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. Journal of the Operational Research Society, 54(6), 627-635.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. NIPS 2017.
- Kaggle – Home Credit Default Risk Dataset. https://www.kaggle.com/c/home-credit-default-risk
- Evidently AI – Data & ML Monitoring. https://evidentlyai.com

MLflow – Open source platform for the machine learning lifecycle. https://mlflow.org